Enhancing Security Camera Systems with LSTM Based Violence Detection

By Lorenzo Benatti (2095334) and Matteo Neri (2109606)

1 - Objective

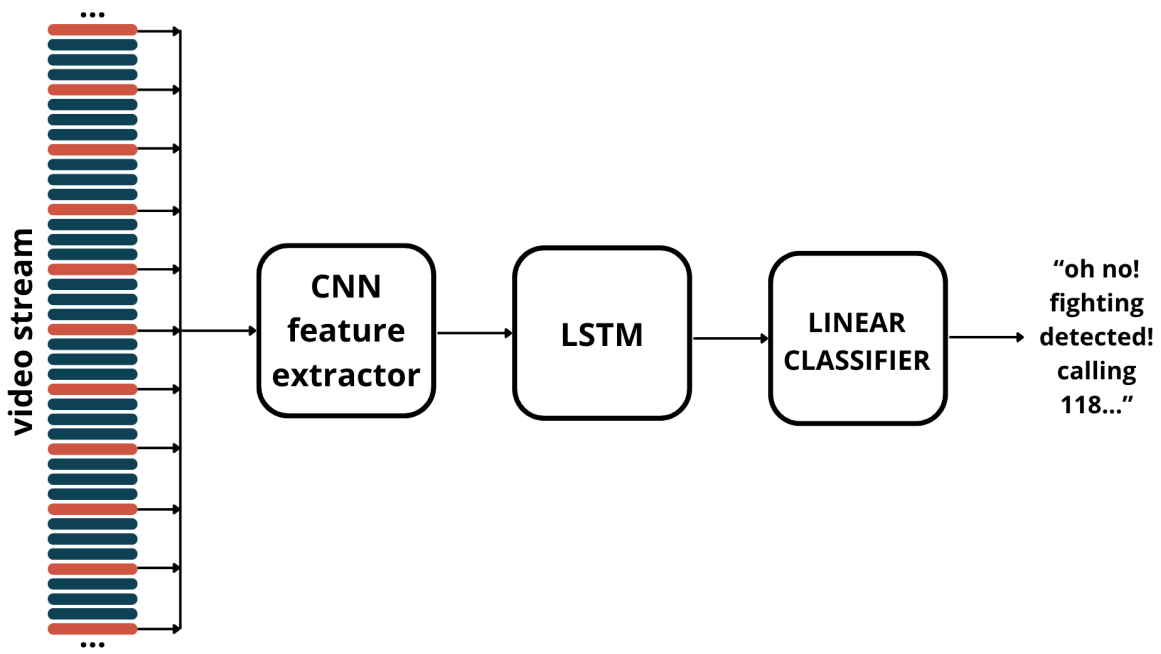
The objective of this project is to design and implement a deep learning based model that can analyze videos and identify violent behavior. The goal is to enable early detection and real-time alerts, thereby improving incident response times and improving public safety.

2 - Model structure

The model follows a the following architecture:

- CNN to extract frame features: every video frame extracted is processed by Resnet-18 to extract spatial features. These features directly represent the visual content and simplify the workload for the rest of the model

- Temporal Modeling using LSTM: The sequence of features is passed to an LSTM network. LSTMs are capable of learning temporal information and are well suited for modeling the evolution of actions over time. This step enables the system to distinguish between similar looking but contextually different actions (ex dancing vs fighting).
- Classification: The final hidden state of the LSTM is passed to a linear layer with softmax activation to get the final result.



3 - Dataset

The dataset is divided into "Violence" videos, which depicts urban fights, aggressive behaviour and physical altercations. On the other hand, "NonViolence" shows peaceful, non threatening encounters.

Violence	1.000 items	Folder
V_1.mp4	3,1 MB	Video
V_2.mp4	3,2 MB	Video
V_3.mp4	1,7 MB	Video
V_4.mp4	2,0 MB	Video
V_5.mp4	1,9 MB	Video
V_6.mp4	2,0 MB	Video
V_7.mp4	11,9 MB	Video
V_8.mp4	14,5 MB	Video
V_9.mp4	5,1 MB	Video

NonViolence	1.000 items	Folder
NV_1.mp4	8,0 MB	Video
NV_2.mp4	530,2 kB	Video
NV_3.mp4	914,0 kB	Video
NV_4.mp4	1,5 MB	Video
NV_5.mp4	1,5 MB	Video
NV_6.mp4	1,4 MB	Video
NV_7.mp4	2,1 MB	Video
NV_8.mp4	1,3 MB	Video
NV_9.mp4	751,4 kB	Video

There are 2000 videos in total, each lasting about 5-10 seconds. The camera can be blurry or on the move, depicting real life cases or movie scenes.

To manage the dataset, a custom class is created. Not only contains the videos but also has methods to transform them and extract features.

In the function *__getitem__*, the video is converted into a fixed-length sequence of frames (64 or 128 frames) linearly extracted using the following algorithm:

```python
def load_video(self, path):
    cap = cv2.VideoCapture(path)
    frames = []
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    sample_rate = max(total // self.NUM_FRAMES, 1)
    count = 0

    while len(frames) < self.NUM_FRAMES and cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        if count % sample_rate == 0:
            frame = cv2.resize(frame, (112, 112))
            frame = frame[:, :, [2, 1, 0]]
            frames.append(frame)
        count += 1
    cap.release()

    # pad with last frame if not enough
    while len(frames) < self.NUM_FRAMES:
        frames.append(frames[-1])  # repeat last frame

    frames = np.stack(frames).astype(np.float32)
    frames = frames.transpose(3, 0, 1, 2)  # C, T, H, W
    frames /= 255.0
    return torch.tensor(frames)
```

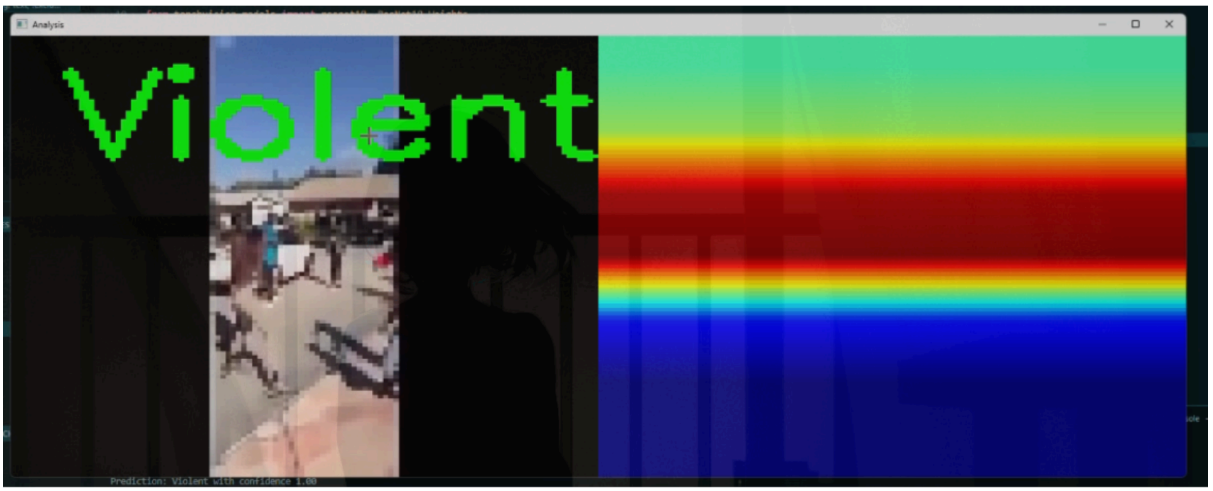
The frames will be extracted at a rate "*sample_rate*", this avoids us having to process the whole video, but still keeps almost all details, as the videos show actions that last the whole duration of the clip, so the probability of missing the key event is very small.

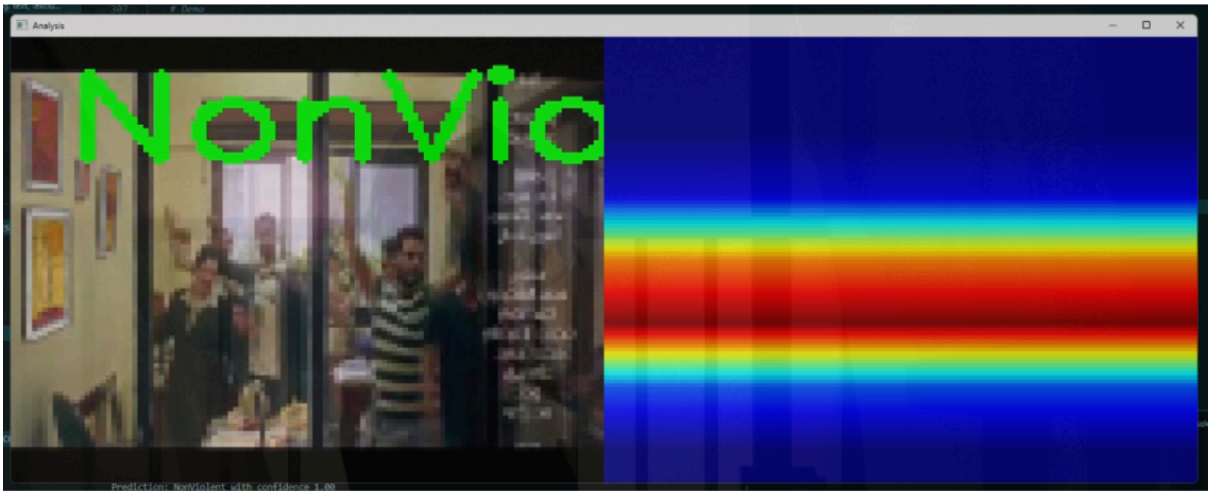
4 - visualization

To show our model in action, we used openCV with Captum.
Captum, a model interpretability library for PyTorch is used for computing gradient-based attribution maps that reveal which regions of each frame contributed the most to the model's decision. By using *LayerGradCam()* we were able to visualize the areas where the model concentrated its attention, highlighted in red. This helped us better understand the inner workings and focus points of the neural network during testing.

5 - Model in action

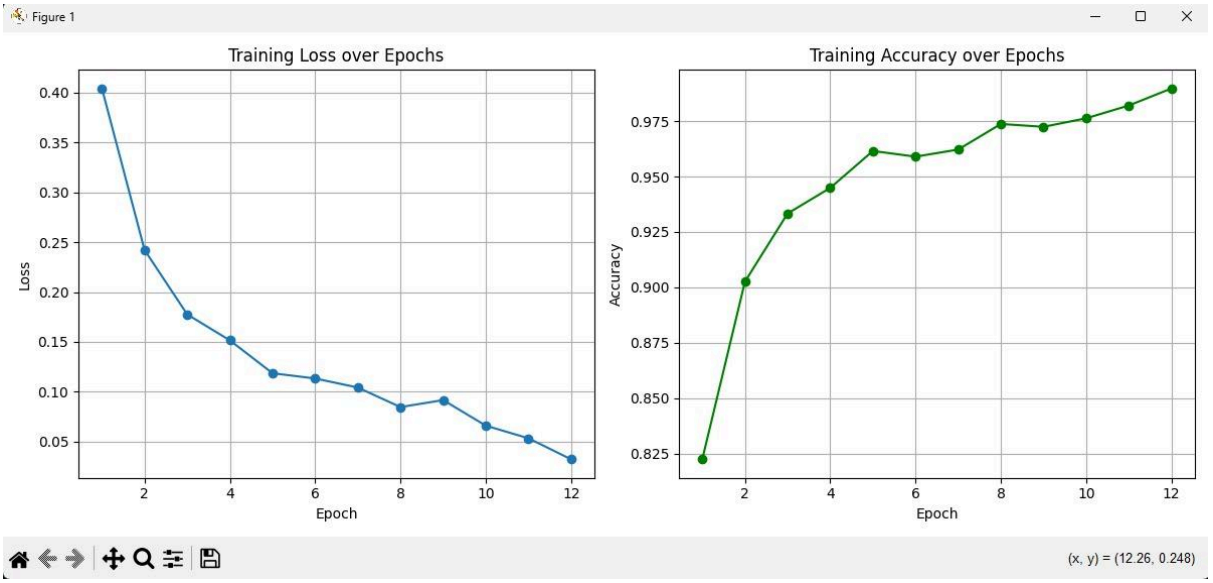
Below is the model in action**.** The scene shows an urban altercation, which the model correctly classifies as "violent" with high confidence. On the right, the gradient map created by Captum highlights the areas of the frame of most importance to the model.





6 - Results

the model reached 99% of accuracy during training with a loss function of ≈0.02



the output of *model.eval()* is as follows:

	precision	recall	f1	support
NonViolent	0.96	0.87	0.92	191
Violent	0.89	0.96	0.93	200
accuracy			0.92	391
macro average	0.92	0.92	0.92	391
weighted average	0.92	0.92	0.92	391

7 - Integration with Security Systems

The core idea was for the model to be integrated into existing security infrastructures.

Alerts can be then forwarded to security personnel or emergency services through a user interface or automated notification system.

8 - Future work

In the future, we aim to extend project with:

- the classification of other classes (verbal aggression, robbery)
- deploy it on a raspberry pi equipped with cameras and servos to steer toward the aggressive behaviour and film it better.